

黄江南

软件开发工程师·后端 (Go) / AI Agent

2027 届·南京邮电大学·求职意向：后端开发 / Agent 开发

3310145612@qq.com

GitHub: JasonBuildAI · 微信: HJN522312MOST

教育背景

南京邮电大学 (双一流) 微电子科学与工程·本科

2023.09 – 2027.06

CET-6 / CET-4 · 核心基础：计算机网络 / 高级语言程序设计 / 操作系统 / 数据库

实习经历

 Apache Casbin (Apache 在孵项目) · Casbin 创始团队 2026 明日之星预选生

2026.04 – 2026.06

导师: hsluoyz (北京大学罗杨教授, Apache Casbin 社区负责人) · 项目: **OpenAgent** (the-open-agent/openagent) —— Casbin 原班人马孵化的独立开源项目, GitHub 5,600+ Stars; 本人为官方 **Member**, 累计代码变更超 **1 万行**, 撰写技术文章累计浏览量超 5 万、带动项目 Star 增长 1,500+。

项目背景: OpenAgent 是面向个人本地部署的开源 AI Agent, 采用 Go 单进程架构, 将 Agent 运行时、工具调用、多模型接入与 Web 控制台收敛为一个预编译二进制: **单文件约 23 MB、冷启动约 2.7 s**, 相较同类 Node.js 方案 (29.2 s、4.2 万 + 文件、382 MB) **快约 11 倍、体积约为其 1/16**。

- **Windows GUI 自动化引擎:** 从 0 到 1 独立设计并交付基于 UIA 的 GUI 工具链, 新增 UIA 引擎、任务调度器与 9 个原子工具 (元素查找 / 交互 / 等待 / 断言 / 应用启动 / 系统指标读取等), 并同步提供非 Windows 平台降级实现与安全熔断。
- **AgentBench 评测框架:** 从 0 到 1 独立设计并交付 Agent 能力基准框架 the-open-agent/agentbench (Python 核心 1692 行 / 33 文件), 覆盖 **8 大套件、104 个任务**, 内置 bootstrap 置信区间统计, 评测可脚本化复现; 据此完成对标 OpenClaw 的横向实测: 工具调用成功率 **83.33% vs 61.11%**、单次耗时 **14.2s vs 26.1s**、Token 消耗 **1312 vs 2362**。
- **会话标题容错链路:** 设计并实现会话标题自动生成与降级链路, 通过 ResolveChatTitle 等函数完成用户消息兜底、HTML 清洗、空白折叠与 rune 安全截断, 保证模型调用失败时标题仍可用。
- **性能、部署与大型特性:** 移除 GetAnswer 接口重复 LLM 调用、补齐 Casdoor 不可用时的默认用量元数据、对齐 docker-compose 端口与数据库主机; 推进动态模型目录与自适应定价 (覆盖 15 个 Provider) 与站点知识自动生成 Skill 两项大型特性。

Casdoor (CNCf 云原生全景图收录·Go + React ·14,000+ Stars) 开源贡献者

2026.04

- 重构并改进项目 README (已合入), 新增简体中文 README 与语言切换入口, 优化文档链接与章节结构; 并参与社区运营与对外推广, 提升中文开发者的上手体验与文档可维护性。

开源经历

 小红书 FireRed-OpenStoryline (Python + JavaScript ·3,385 Stars / 394 Forks) 核心贡献者

2026.02 – 2026.04

项目背景: 小红书 FireRedTeam 开源的 AI 视频剪辑 Agent, 以自然语言交互 + LLM 规划 + 工具编排把手工剪辑转变为“意图驱动”的创作流程; 本人在所有贡献者中 **PR 数第二**, 独立承担会话状态持久化与恢复、会话历史、媒体链路与可用性治理等主干模块。

- **会话状态持久化与安全恢复:** 独立设计并实现会话状态落盘与恢复机制——对损坏的工具调用行采取 fail-closed 策略而非静默跳过; 为会话历史与 LangChain 消息分别设置 2000 / 4000 条上限; 构建递归密钥脱敏链路, 按字段名与内联赋值两种模式识别 client_secret / refresh_token / x-api-key / access_key 等敏感字段并统一替换为 ***; 对 getSession 异常做类型化处理。
- **会话历史与媒体链路:** 独立完成 Web 端对话历史侧栏 (面板 DOM / 样式 / 中英双语 i18n / aria 无障碍标签), 基于 localStorage 实现会话 ID 与列表双键存储、历史会话渲染切换与切换保护; 后端抽象 MCP 媒体传输策略 inline_media = auto / always / never, 按 connect_host 自动判定同机与跨机链路, 同机路径直传、跨机才内联 base64, 避免远程模式下 base64 放大导致的超大 payload。
- **Agent 可用性与稳定性治理:** 新增 API Key 前置校验, 任务开始前向 OpenAI 兼容接口发一次最小请求 (max_tokens=1 控制成本、10s 超时), 区分 401 / 404 / 429 / 5xx / 超时 / 连接失败并返回可操作提示; 为媒体搜索与下载补齐指数退避重试 (初始 1s、2 倍退避、上限 10s), 区分网络错误与非网络错误; 统一日志出口并完善媒体文件保存的错误处理。

项目经历

AegisNest 智能客服系统 (Python + FastAPI + LangGraph) 个人项目

2026.07 – 2026.09

项目介绍: 面向 API 开放平台的 Multi-Agent 智能客服系统, 以 LangGraph 的 DAG 编排协作为核心, 集成意图路由、RAG 混合检索、工具诊断、会话记忆、安全脱敏、多级兜底等能力, 为开发者提供全流程智能技术支持。

- **Multi-Agent 编排:** 基于 LangGraph StateGraph 设计意图路由、文档 RAG、API 诊断、账单、工单、安全审查 6 个专业 Agent 的职责划分与 DAG 管线, 由中心编排器统一调度, 安全审查节点串行收口, 保证安全审查一定在最终输出之前执行。
- **RAG 混合检索:** 实现向量检索 (text-embedding-v3, 1024 维) + BM25 双路并行召回, 经 RRF 融合、gte-rerank-v2 精排、上下文压缩后带引用生成; 为错误码类问题设计标量热路径直查数据库, 该环节耗时从 **0.74s 降到 0.010s (约 78 倍)**。
- **性能优化:** 设计语义缓存、路由缓存与模型分层, 叠加错误码热路径和 Multi-Agent 并行策略, 冷启动 P95 从 **48.58s 降到 6.83s (约 7 倍)**, 吞吐从 **0.18 提升到 0.75 req/s**, 热点问题命中缓存仅 **0.38s、零 token 消耗 (约 26 倍加速)**。
- **安全、兜底与可观测:** 实现覆盖 API Key、Token、手机号等八类敏感信息的三层脱敏 (用户输入、工具日志、最终输出), 安全审查节点在管线末端收口; 多级兜底覆盖从单 Agent 异常到整条管线崩溃的各种失败; 基于逐节点链路追踪实现全链路可观测与按租户成本统计。

MockAtlas 智能面试助手 (Go + Eino ·Multi-Agent) 个人项目

2025.12 – 2026.02

项目介绍: 基于 Go 与 Eino 框架的 AI 智能面试助手, 以 Multi-Agent 的 DAG 编排协作为核心, 集成 RAG 多路召回题库检索、动态难度调节、Agent 记忆系统等能力, 提供简历分析、智能出题、实时面试、评估报告的全流程模拟面试体验。

- **Multi-Agent 编排:** 设计 7 个 Agent 的职责划分与 7 阶段 DAG 编排流程, 采用中央编排器模式管理 Agent 间数据流转, 实现从 JD 分析、简历匹配、出题规划到面试评估的全链路自动化。
- **RAG 多路召回:** 实现 Milvus 向量检索 (1024 维) + 纯内存 BM25 关键词检索双路并行召回, 通过 RRF(k=60) 融合排序 + LLM Rerank 重排序输出最终候选题, 支持按 user_id 字段的用户级数据隔离。
- **记忆、难度调节与 Skill:** 实现滑动窗口短期记忆 + Redis/MySQL 长期用户画像存储与薄弱知识点自动追踪; 采用两阶段出题 + 动态难度调节 + 智能追问实现难度自适应; 设计有状态多轮交互 Skill 接口 (区别于无状态 Tool), 开发快速测验、概念教学等 4 个内置 Skill, 通过优先级匹配的 SkillRegistry 统一管理。
- **RAG 质量评估与调优:** 构建离线评估流水线, 基于 50 条标注样本计算 Recall@K / MRR 指标并按 topic 分组统计; 通过 A/B 对比将 TopK 从 10 调至 20, **Recall@10 提升 7.3%、MySQL 领域召回率提升 16.7%**。

个人技术栈

- **技术栈:** Go、Python、TypeScript / JavaScript、C; MySQL、Redis、消息队列、RESTful API; LangChain / LangGraph / Eino、RAG (向量检索 / BM25 / RRF / Rerank)、Tool Calling、MCP / A2A; Docker、Linux、Git、React / Vue。